עזריאלי
מכללה אקדמית להנדסה
ירושלים

Memory Game
Vulnerable vs Secure Implementation

דוחא פרחאן – Duha Farhan
סמירה אבו אלהוא – Samira Abu Elhawa
הבה עבדו – Hiba Abdo

הנדסת תוכנה

קורס אבטחת מערכות WEB

המכללה האקדמית להנדסה עזריאלי ירושלים
Azrieli College of Engineering Jerusalem

- תוכן עניינים:

תיאור החולשה והרקע התיאורטי 2
מהי החולשה 2
כיצד היא נוצרת 2
באיזה הקשר היא מופיעה במערכות ווב 3
מה תוקף יכול להשיג באמצעותה 3
סביבת העבודה 3
מימוש והדגמת המתקפה ((POC 4
1) מתקפה SQL Injection - Login Bypass 4
תיאור ה־POC 4
שלבי הניצול 4
תוצאה בפועל – גרסה פגיעה 5
תוצאה בפועל – גרסה מאובטחת 5
דיאגרמה המתארת את שלבי המתקפה - גרסה פגיעה 6
דיאגרמה המתארת את שלבי המתקפה - גרסה מאובטחת 6
2) מתקפה IDOR Score Manipulation 7
תיאור ה־POC 7
שלבי הניצול 7
תוצאה בפועל – גרסה פגיעה 7
תוצאה בפועל – גרסה מאובטחת 8
דיאגרמה המתארת את שלבי המתקפה - גרסה פגיעה 10
דיאגרמה המתארת את שלבי המתקפה - גרסה מאובטחת 10
3) מתקפה Stored XSS 11
תיאור ה־POC 11
שלבי הניצול 11
תוצאה בפועל – גרסה פגיעה 11
תוצאה בפועל – גרסה מאובטחת 12
דיאגרמה המתארת את שלבי המתקפה - גרסה מאובטחת 14
מנגנוני הגנה ומניעה 14
מדוע ההגנות נכשלו בגרסה הפגיעה 15
כיצד ניתן למנוע את החולשה 15
Best Practices רלוונטיים 15
מקורות 16
סיכום 16

תיאור החולשה והרקע התיאורטי

במסגרת פרויקט זה פותחה אפליקציית Web בשם Memory Game, אשר מומשה בשתי גרסאות נפרדות:

- גרסה פגיעה (Vulnerable)
- גרסה מאובטחת (Secure)

מטרת הפרויקט הייתה להדגים חולשות אבטחה נפוצות במערכות Web, לבצע ניצול בפועל (POC), ולאחר מכן ליישם מנגנוני הגנה ולבחון את יעילותם.

בפרויקט הודגמו שלוש חולשות מרכזיות מתוך OWASP Top 10:

1. SQL Injection
2. Insecure Direct Object Reference - IDOR
3. Stored Cross-Site Scripting- Stored XSS

מהי החולשה

1) SQL Injection

חולשה הנוצרת כאשר שאילתות למסד נתונים נבנות באמצעות שרשור מחרוזות במקום שימוש בפרמטרים מאובטחים (Prepared Statements).
תוקף יכול לשנות את לוגיקת השאילתה ולבצע עקיפת אימות.

2) IDOR

חולשה שבה השרת סומך על מזהים שמגיעים מהלקוח (כגון userId) מבלי לבדוק הרשאות מתאימות.
מאפשר גישה או שינוי נתונים של משתמש אחר.

3) Stored XSS

חולשה שבה קלט משתמש נשמר במסד הנתונים ומוצג בדפדפן ללא סניטציה.
מאפשר הרצת קוד JavaScript זדוני בדפדפן של משתמשים אחרים.

כיצד היא נוצרת

- בניית שאילתות SQL באמצעות String Concatenation
- אמון בקלט משתמש בצד השרת
- היעדר בדיקות הרשאה
- שימוש ב-inner HTML ללא escaping

באיזה הקשר היא מופיעה במערכות ווב

- טפסי התחברות (Login)
- מערכות ניקוד
- מערכות תגובות
- פורומים
- Dashboards

מה תוקף יכול להשיג באמצעותה

- עקיפת אימות משתמש
- התחברות ללא סיסמה
- שינוי ניקוד של משתמשים אחרים
- פגיעה בשלמות הנתונים
- הרצת קוד זדוני בדפדפן
- גניבת Session (בתרחיש אמיתי)

סביבת העבודה

Backend:

- Node.js
- Express.js
- SQLite Database
- express-session

Frontend:

- HTML5
- CSS3
- Vanilla JavaScript
- Fetch API

כלי הדגמה:

- Postman לביצוע מתקפות
- Chrome Developer Tools
- GitHub ניהול גרסאות
- שני Branches

  feat/backend-vuln

  feat/backend-secure
- המערכת רצה על:
  http://localhost:3000

מימוש והדגמת המתקפה (POC)

1) מתקפה SQL Injection - Login Bypass

תיאור ה־POC

בגרסה הפגיעה, שאילתת ה־SQL נבנתה בצורה לא מאובטחת ולכן ניתן היה להחדיר תנאי לוגי שמשנה את תוצאת השאילתה.

שלבי הניצול

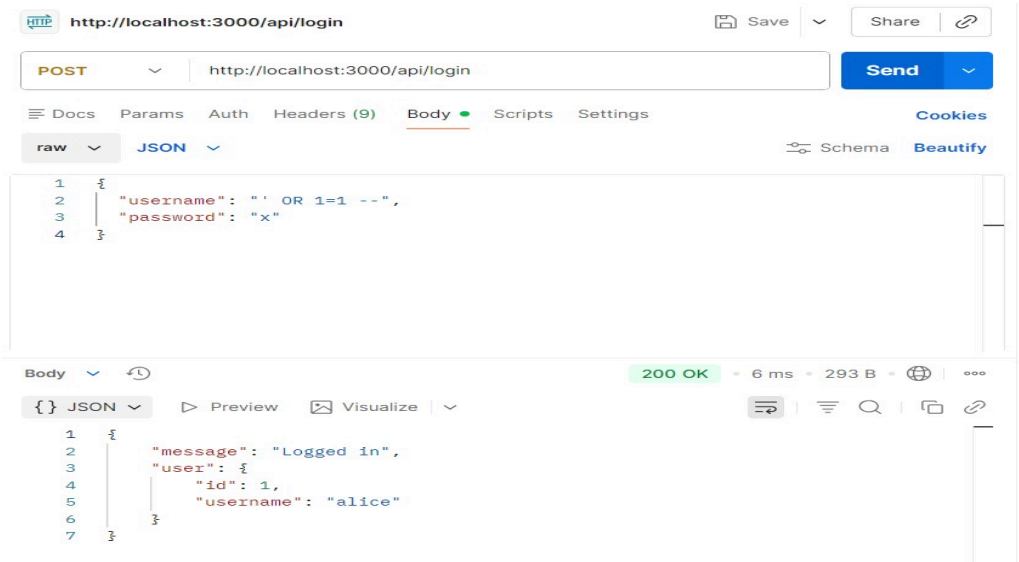
1. שליחת בקשה POST אל: /api/login
2. שימוש ב- Payload זדוני:

```
{
"username" : " ' OR 1=1-- ",
"password": "x"
}
```

3. השרת בונה שאילתה לוגית שבה התנאי 1=1 תמיד מתקיים.
4. מתבצעת התחברות ללא בדיקת סיסמה אמיתית.

תוצאה בפועל – גרסה פגיעה

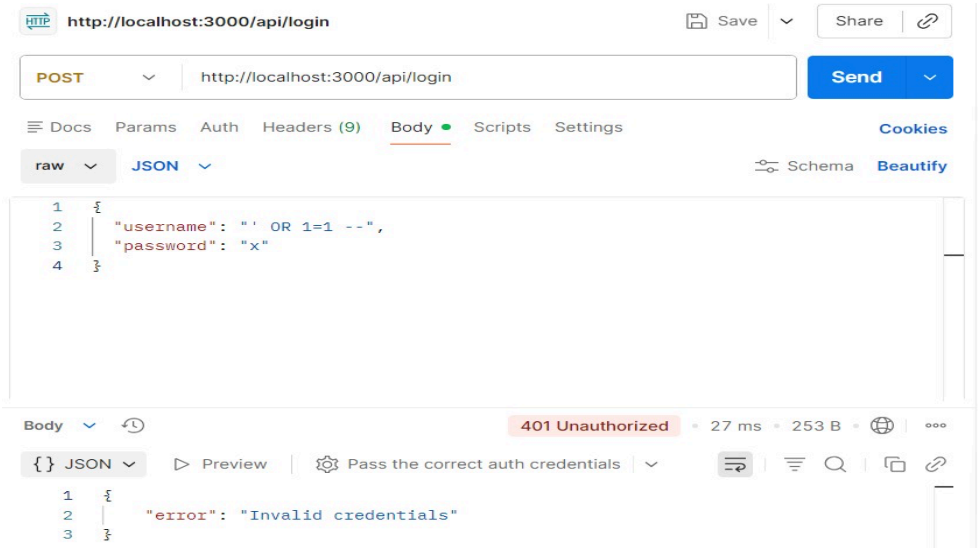
השרת החזיר 200 OK. המשתמש התחבר כמשתמש הראשון במסד הנתונים.



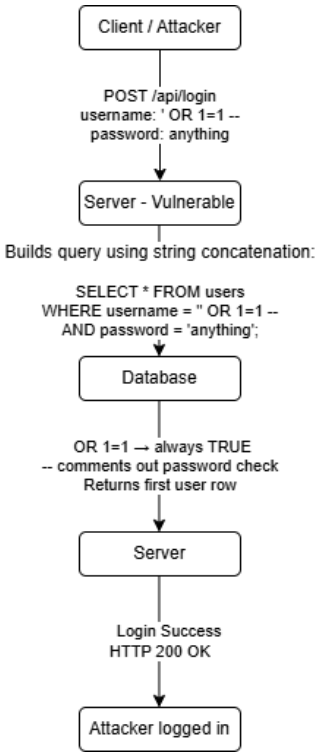
תוצאה בפועל – גרסה מאובטחת

בגרסה המאובטחת נעשה שימוש ב־Parameterized Queries.

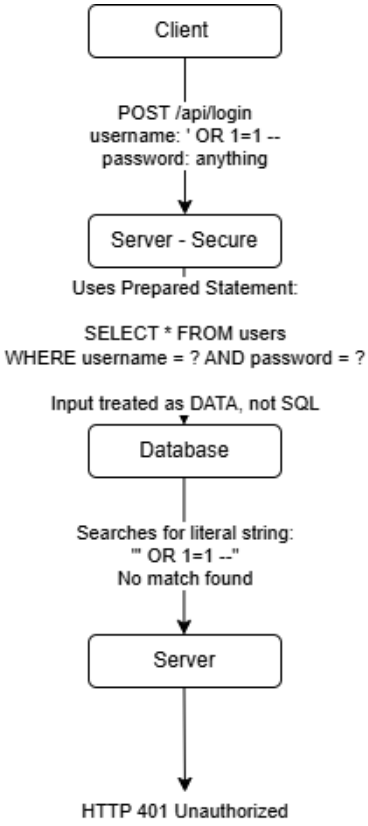
השרת החזיר:Unauthorized 401



דיאגרמה המתארת את שלבי המתקפה - גרסה פגיעה



דיאגרמה המתארת את שלבי המתקפה - גרסה מאובטחת



2) מתקפה IDOR Score Manipulation

תיאור ה־POC

בגרסה הפגיעה השרת קיבל userId מהלקוח ללא אימות הרשאות.
לא בוצעה בדיקה שהמשתמש המחובר אכן מורשה לעדכן את הניקוד של אותו userId

---

שלבי הניצול

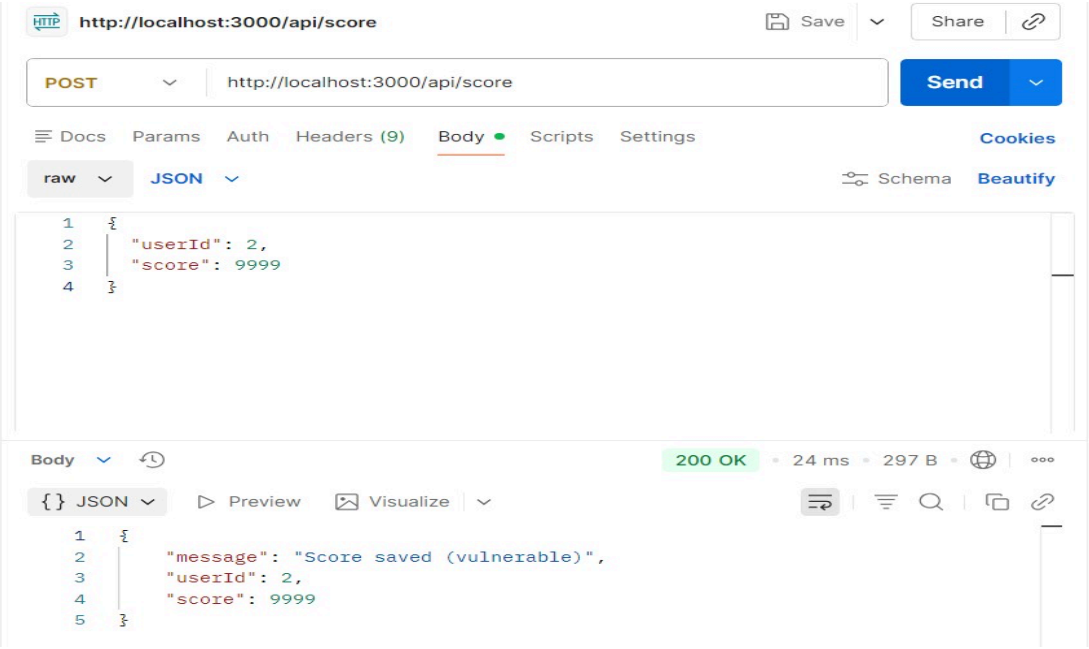
1. התחברות כמשתמש אחד.
2. שליחת בקשה ל- /api/score עם userId של משתמש אחר.
3. שמירת ניקוד גבוה עבור משתמש אחר.

Payload לדוגמה:

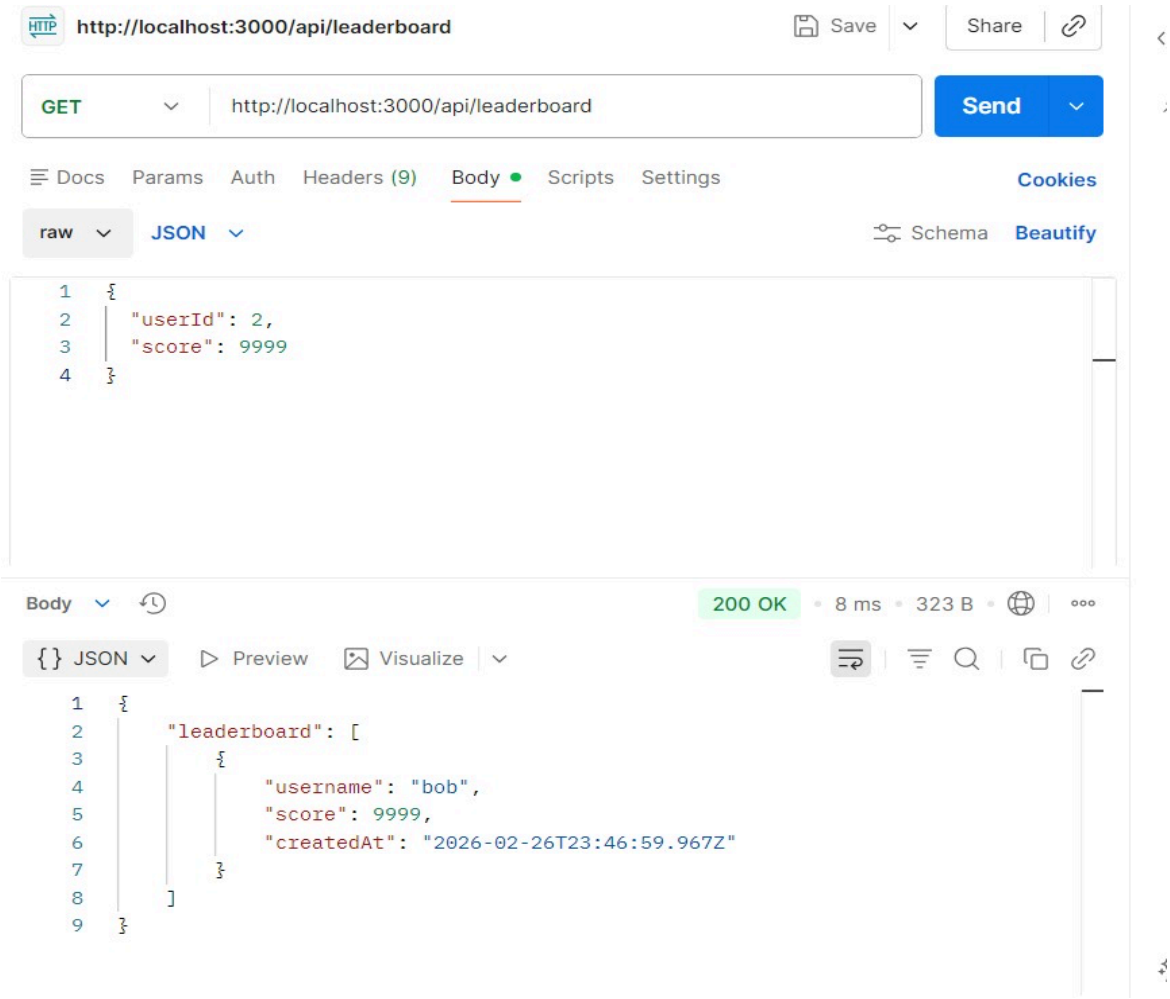
```
{
"userId" : 2,
"Score": 9999
}
```

תוצאה בפועל – גרסה פגיעה

איור 1 – שמירת ניקוד עבור userId: 2 בגרסה הפגיעה. ניתן לראות כי השרת החזיר 200 OK ואישר שינוי ניקוד עבור משתמש אחר.



איור 2 – לוח התוצאות לאחר הניצול.
ניתן לראות כי הניקוד 9999 נשמר והוצג עבור המשתמש bob.

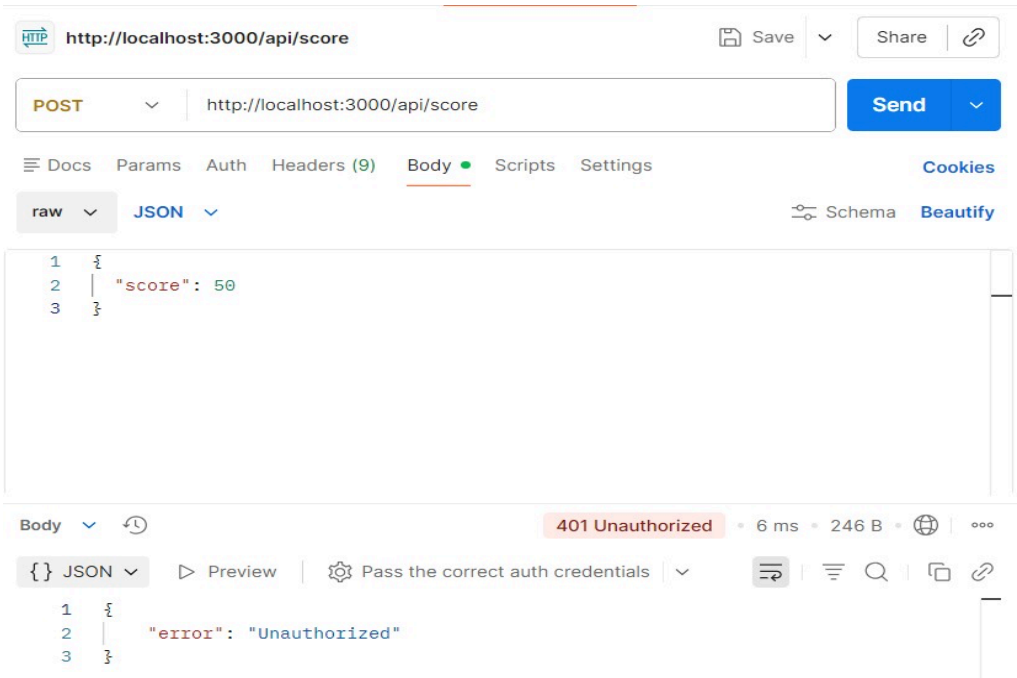


תוצאה בפועל – גרסה מאובטחת

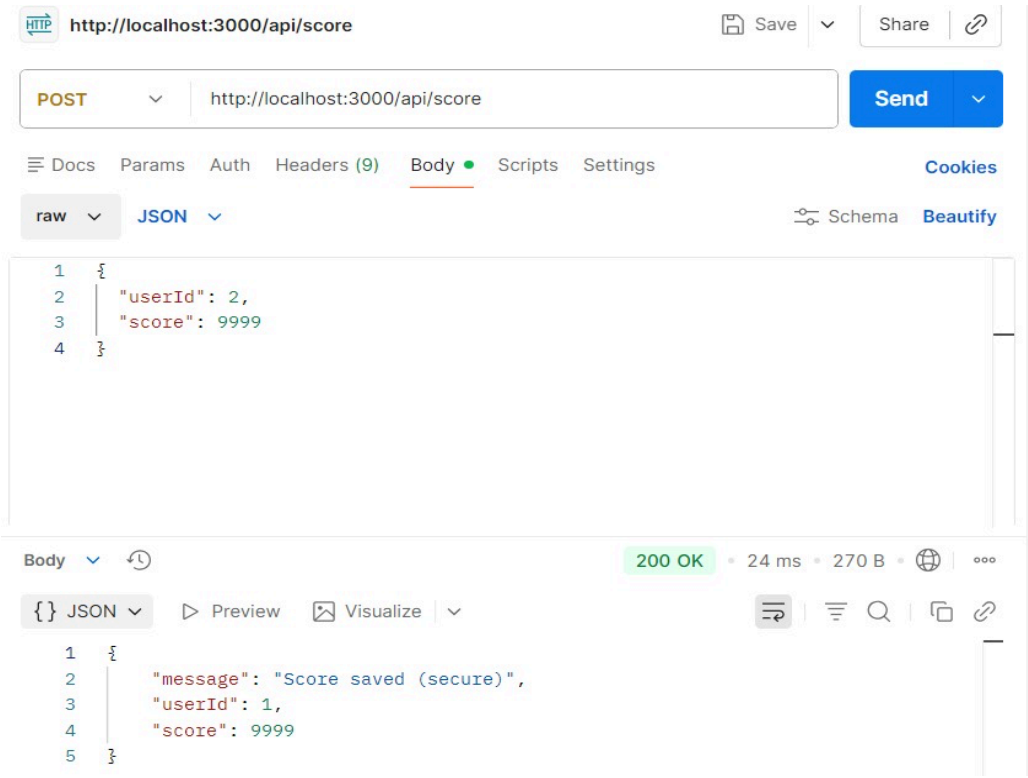
בגרסה המאובטחת:

- השרת מתעלם מה-userId שנשלח
- משתמש ב-userId מתוך ה-Session בלבד
- או מחזיר 401 אם אין התחברות.

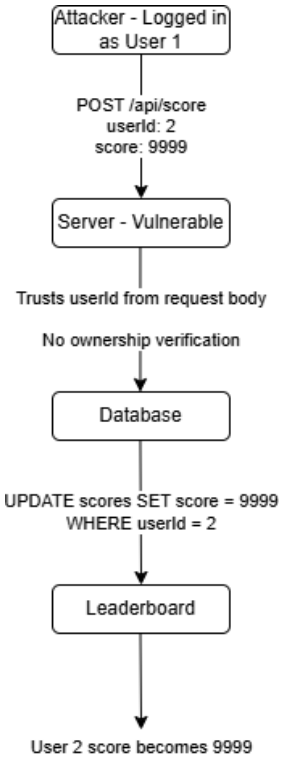
איור 1 – השרת מחזיר Unauthorized 401 כאשר אין Session פעיל.
לא ניתן לשמור ניקוד ללא אימות משתמש.



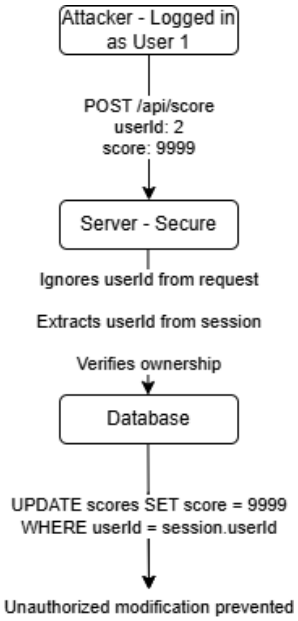
איור 2 – השרת מתעלם מה-userId שנשלח מהלקוח
ומשתמש ב-userId מתוך ה-Session בלבד.



דיאגרמה המתארת את שלבי המתקפה - גרסה פגיעה



דיאגרמה המתארת את שלבי המתקפה - גרסה מאובטחת



3) מתקפה Stored XSS

תיאור ה־POC

השרת שמר תגובה הכוללת קוד JavaScript זדוני והציג אותה בדפדפן ללא סינון.
לא בוצע סינון קלט (Input Validation) או ניקוי פלט (Output Encoding).

שלבי הניצול

1. שליחת בקשה ל- /api/comment עם Payload זדוני:

```
{
"userId" : 1,
"content": "<script>alert('XXS')</script>"
}
```

2. שמירת התגובה במסד הנתונים.

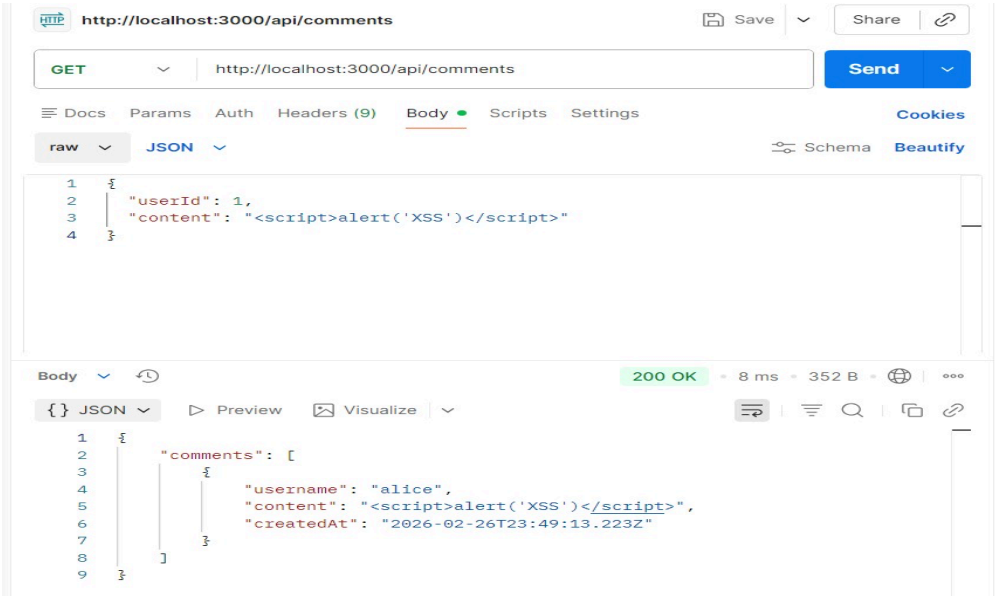
3. טעינת התגובות בדפדפן.

4. הרצת הקוד הזדוני בצד הלקוח.

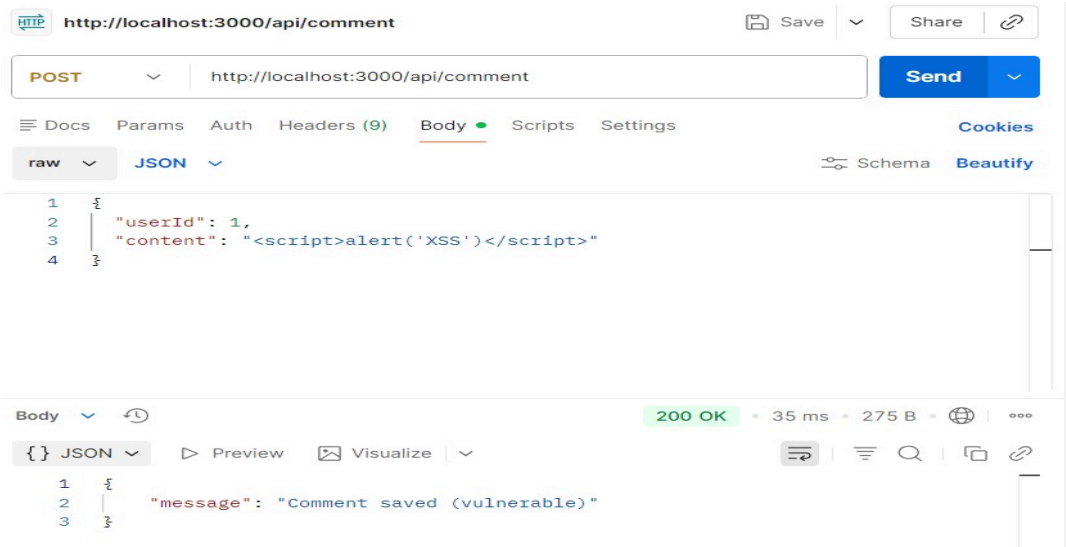
תוצאה בפועל – גרסה פגיעה

התגובה נשמרה והקוד הופיע במסד הנתונים.

איור 1 – השרת קיבל ושמר תגובה המכילה קוד <script>alert('XSS')</script>

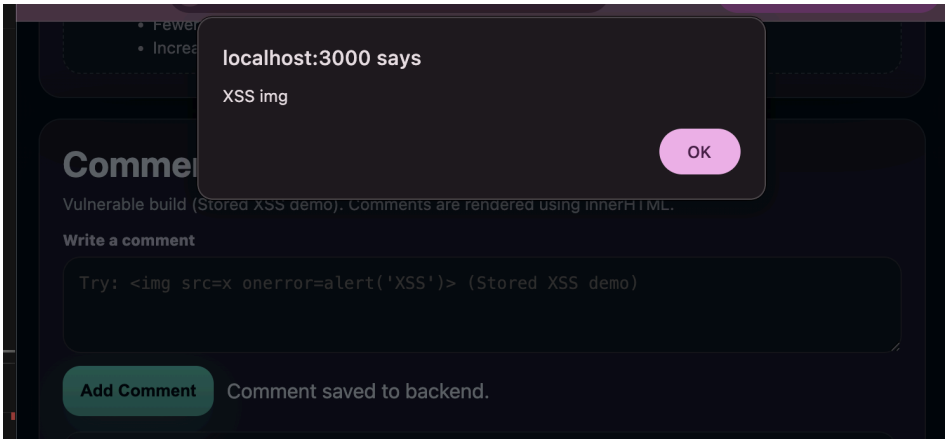


איור 2 – בעת שליפת התגובות מהשרת, הקוד הזדוני, נשבר ומוחזר למשתמשים אחרים ללא סינון.



איור 3 – הקוד הזדוני הופעל בצד הלקוח לאחר שליפת התגובה מהשרת.

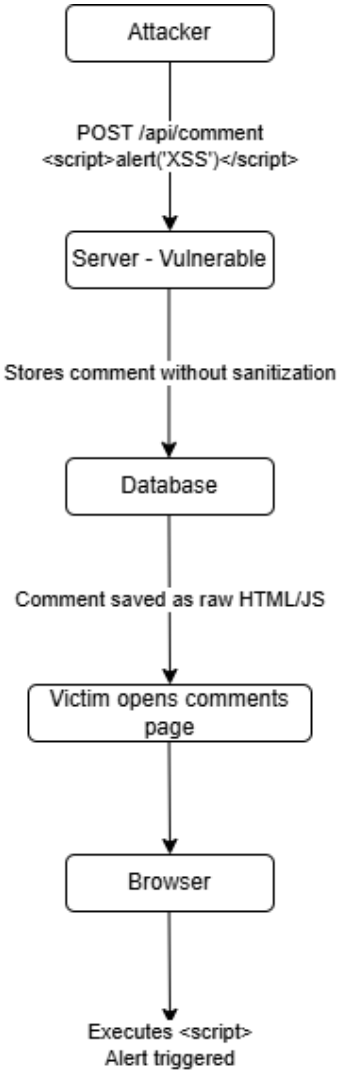
מדובר בפגיעות מסוג Stored XSS, שכן הקוד נשמר במסד הנתונים ומבוצע בכל טעינת הדף.



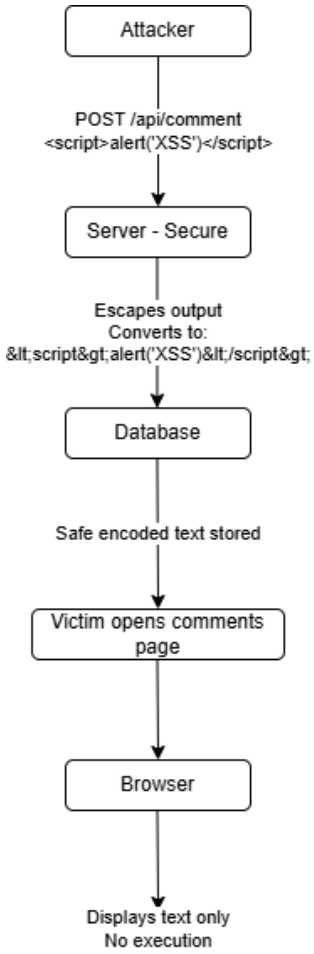
תוצאה בפועל – גרסה מאובטחת

בגרסה המאובטחת:
התוכן עובר escaping / sanitization
הקוד מוצג כטקסט בלבד ואינו מבוצע.

דיאגרמה המתארת את שלבי המתקפה - גרסה פגיעה



דיאגרמה המתארת את שלבי המתקפה - גרסה מאובטחת



מנגנוני הגנה ומניעה

בגרסה המאובטחת יושמו:

- שימוש ב-Prepared Statements
- אימות זהות באמצעות Session
- בדיקת הרשאות בצד השרת בלבד
- מניעת אמון בקלט מהלקוח
- החזרת HTTP Status Codes תקינים
- הפרדה ברורה בין גרסה פגיעה לגרסה מאובטחת

מדוע ההגנות נכשלו בגרסה הפגיעה

- בניית שאילתות SQL לא מאובטחות
- היעדר בדיקות הרשאה
- אמון מלא בקלט משתמש
- חוסר סניטציה

כיצד ניתן למנוע את החולשה

1. מתקפה SQL Injection - Login Bypass שימוש ב־**Prepared Statements (שאילתות פרמטריות)** במקום חיבור מחרוזות ישיר לשאילתה.
   - ולידציה לקלט בצד השרת.
   - הגבלת הרשאות במסד הנתונים.
   - טיפול בשגיאות מבלי לחשוף מידע רגיש
2. מתקפה IDOR Score Manipulation
   - שימוש ב־**Session Management**, כאשר מזהה המשתמש נלקח מה־session בשרת ולא לסמוך על מזהה משתמש שמגיע מהלקוח.
   - בדיקות הרשאה בכל בקשה רגישת.
   - אימות שהמשתמש מחובר לפני ביצוע פעולה.
   - החזרת קוד שגיאה במקרה של ניסיון לא מורשה.
3. מתקפה Stored XSS
   - ביצוע **סניטיזציה (Sanitization)** לקלט מהמשתמש.
   - שימוש ב־**textContent** במקום **innerHTML** בעת הצגת נתונים בדפדפן.
   - סינון תגיות **HTML** מסוכנות.
   - הוספת מדיניות אבטחה בדפדפן (Content Security Policy).

Best Practices רלוונטיים

- Never trust client input
- Always use parameterized queries
- Enforce authorization server-side
- Validate and sanitize user input
- Follow OWASP Top 10

מקורות

OWASP – SQL Injection
https://owasp.org/www-community/attacks/SQL_Injection

OWASP – XSS
https://owasp.org/www-community/attacks/xss

OWASP – IDOR
https://owasp.org/www-community/attacks/Insecure_Direct_Object_Reference

PortSwigger Web Security Academy
https://portswigger.net/web-security

GitHub Repository:
https://github.com/Azrieli-College-of-Engineering/final-project-duha-samira-and-hiba

---

סיכום

הפרויקט מדגים בצורה מעשית כיצד טעויות תכנות בסיסיות גורמות לפרצות אבטחה חמורות במערכות Web.

באמצעות בניית שתי גרסאות נפרדות ניתן היה:

- לזהות חולשות
- לנצל אותן בפועל
- ליישם תיקונים
- לוודא המתקפות נחסמות

הפרויקט משקף תהליך עבודה אמיתי בתחום אבטחת המידע:
זיהוי ← ניצול ← תיקון ← אימות